

MODEL PARALELISASI MENGGUNAKAN *DIVIDE AND CONQUER*, PIPELINE, DAN MAP-REDUCE

Bayu Endru Subiyakto¹, Fawwaz Muzakty², Alfi Muiz³, Muhammad Faiz Fairuz⁴

Universitas Islam Negeri Sultan Maulana Hasanuddin Banten ^{1,2,3,4}

Email: bayuendru08@gmail.com¹, alfimuiz5@gmail.com², fawazmuzaki3@gmail.com³,
Faizfairuz2005@gmail.com⁴

Informasi	Abstract
Volume : 3 Nomor : 1 Bulan : Januari Tahun : 2026 E-ISSN : 3062-9624	<i>This study discusses a computational parallelization model that integrates the divide and conquer, pipeline processing, and map-reduce approaches as strategies to improve data processing efficiency and the performance of modern computing systems. The background of this study is based on the growing need to optimize increasingly complex computational processes, particularly in large-scale data processing scenarios and applications that require fast execution times. The literature review covers the fundamental concepts, functions, and main roles of each parallel algorithm in creating more efficient processing structures. The research method adopts a qualitative approach with descriptive analysis, focusing on the interpretation of literature, mapping of operational mechanisms, and comparative evaluation of the three parallelization models within the context of practical implementation. The results indicate that combining the three models can enhance scalability, reduce bottlenecks, and significantly accelerate computation time. This study confirms that the appropriate selection and integration of parallel strategies can effectively support the demands of modern computing in both distributed environments and multi-core architectures.</i>

Keyword: Parallel Algorithms, Divide and Conquer, Pipeline, Map-Reduce

Abstrak

Studi ini membahas model paralelisasi komputasi yang mengintegrasikan pendekatan divide and conquer, pipeline processing, dan map-reduce sebagai strategi untuk meningkatkan efisiensi pengolahan data dan performa sistem komputasi modern. Latar belakang kajian didasarkan pada kebutuhan optimasi proses komputasi yang semakin kompleks, terutama pada skenario pengolahan data besar dan aplikasi yang menuntut waktu eksekusi cepat. Tinjauan pustaka mencakup konsep dasar, fungsi, serta peran utama masing-masing algoritma paralel dalam menciptakan struktur pemrosesan yang lebih efisien. Metode penelitian mengadopsi pendekatan kualitatif dengan analisis deskriptif yang berfokus pada interpretasi literatur, pemetaan mekanisme kerja, serta evaluasi komparatif terhadap ketiga model paralelisasi tersebut dalam konteks penerapan aktual. Hasil pembahasan menunjukkan bahwa kombinasi ketiga model dapat meningkatkan kemampuan skalabilitas, mengurangi bottleneck, dan mempercepat waktu komputasi secara signifikan. Studi ini menegaskan bahwa pemilihan dan penggabungan strategi paralel yang tepat mampu mendukung kebutuhan komputasi modern pada lingkungan terdistribusi maupun arsitektur multi-inti.

Kata Kunci: Algoritma Pararel, Divide And Conquer, Pipeline, Map-Reduce

A. PENDAHULUAN

Paralelisasi komputasi telah menjadi kebutuhan fundamental dalam sistem komputasi modern karena meningkatnya kompleksitas data dan beban pemrosesan. Pertumbuhan data yang bersifat *massive-scale* menuntut model komputasi yang tidak hanya cepat tetapi juga mampu mendistribusikan tugas secara efisien ke berbagai unit pemrosesan. Dalam konteks ini, strategi paralelisasi seperti *divide and conquer*, *pipeline*, dan *map-reduce* berkembang sebagai pendekatan yang relevan untuk mengatasi permasalahan komputasi berskala besar. Ketiga pendekatan tersebut menawarkan mekanisme pemrosesan berbeda tetapi saling melengkapi, sehingga memberikan fleksibilitas dalam desain solusi komputasi berbasis multi-core maupun *distributed systems*.

Perkembangan perangkat keras seperti prosesor multi-core, GPU, dan kluster komputasi mendorong kebutuhan akan model algoritmik yang dapat memanfaatkan paralelisme tersebut secara optimal. Tanpa strategi paralelisasi yang tepat, peningkatan kapasitas perangkat keras tidak akan secara otomatis menghasilkan peningkatan performa komputasi. Oleh karena itu, studi mengenai model paralelisasi menjadi semakin penting, terutama ketika organisasi dan peneliti menghadapi tantangan pengolahan data real-time, pemrosesan visual, analisis numerik, maupun pemrosesan *big data*. Keselarasan antara desain algoritma dan arsitektur eksekusi menjadi titik krusial yang menentukan efektivitas implementasi paralel.

Salah satu tantangan terbesar dalam merancang algoritma paralel adalah menentukan bagaimana tugas dipecah, didistribusikan, dan disinkronkan. Tidak semua permasalahan dapat diparalelkan secara sempurna; beberapa memiliki ketergantungan data yang menghambat eksekusi paralel. Oleh karena itu, memilih model paralelisasi yang sesuai dengan karakteristik masalah merupakan langkah strategis. *Divide and conquer* cocok untuk masalah rekursif independen, *pipeline* untuk proses berurutan bergaya alur produksi, dan *map-reduce* sangat efektif untuk pemrosesan data masif yang dapat dipecah menjadi pasangan *key-value*. Penyesuaian model terhadap pola dependensi data menjadi kunci keberhasilan.

Model *divide and conquer* menjadi pendekatan yang sangat dikenal dalam desain algoritma karena strategi pemecahan masalahnya yang sistematis. Dengan membagi masalah besar menjadi sub-masalah kecil yang dapat diproses secara independen, pendekatan ini memberikan peluang besar untuk eksekusi paralel yang terstruktur. Banyak algoritma klasik seperti *merge sort*, *quick sort*, maupun pencarian dalam ruang solusi kompleks menggunakan metode ini. Keunggulannya terletak pada sifat rekursif yang mudah diterjemahkan ke lingkungan multi-threading maupun *task scheduling* modern.

Berbeda dengan *divide and conquer*, model *pipeline* lebih cocok untuk pemrosesan yang berlangsung secara bertahap. Pada pendekatan ini, data mengalir melalui serangkaian tahap yang masing-masing dapat berjalan paralel dengan tahap lainnya. Model ini menyerupai jalur produksi industri, di mana setiap tahapan memiliki tanggung jawab khusus dan bekerja secara simultan dengan tahap lain. Pendekatan *pipeline* sering dimanfaatkan dalam pemrosesan citra, kompilasi program, hingga sistem *data streaming* yang membutuhkan alur kontinu dan waktu respons cepat.

Sementara itu, *map-reduce* menjadi model paralelisasi yang mendapat perhatian besar sejak kemunculan paradigma *big data*. Pendekatan ini berfokus pada pemrosesan data dalam skala yang sangat besar dengan cara memetakan data ke pasangan *key-value* lalu menggabungkannya. Model ini memungkinkan distribusi pemrosesan ke ribuan node secara efisien, sehingga sangat berpengaruh dalam ekosistem Hadoop, Spark, dan berbagai platform komputasi terdistribusi. Keunggulannya terletak pada skalabilitas horizontal, toleransi terhadap kesalahan, serta kemampuannya menangani *unstructured data* secara efektif.

Studi mengenai ketiga model paralelisasi tersebut penting karena setiap pendekatan memiliki ruang aplikasi dan efektivitas yang berbeda. Dengan memahami karakteristik masing-masing model, peneliti dan pengembang dapat menyesuaikan strategi paralel sesuai kebutuhan komputasi. Pengembangan aplikasi komputasi modern tidak hanya mengandalkan teori algoritmik, tetapi juga pada kemampuan untuk mengintegrasikan model paralel secara optimal dalam implementasi praktis. Pengetahuan mengenai kesesuaian model terhadap jenis beban kerja menjadi kompetensi yang sangat diperlukan di era komputasi kontemporer.

Selain itu, analisis komparatif antar model paralelisasi memberikan wawasan mengenai keunggulan dan keterbatasan tiap pendekatan. Hal ini penting terutama dalam konteks pengembangan sistem yang membutuhkan efisiensi tinggi, seperti aplikasi sains komputasi, kecerdasan buatan, dan analitik data berskala besar. Pemahaman mendalam mengenai cara kerja dan performa model membantu dalam memilih solusi yang paling efisien, sekaligus mendorong peluang inovasi dalam perancangan algoritma paralel generasi berikutnya.

Kebutuhan akan komputasi paralel yang semakin besar juga dipengaruhi oleh meningkatnya permintaan terhadap layanan digital berbasis data. Sistem informasi modern membutuhkan waktu pemrosesan yang cepat untuk mendukung keputusan strategis, prediksi, dan analisis berbasis data. Kondisi ini membuat organisasi tidak hanya fokus pada penyediaan perangkat keras yang kuat, tetapi juga pada optimalisasi model algoritmik. Oleh karena itu,

desain paralelisasi yang tepat bukan sekadar aspek teknis, tetapi juga aspek strategis dalam pengembangan teknologi.

Penelitian mengenai model paralelisasi juga memberikan kontribusi penting dalam optimalisasi konsumsi energi, karena eksekusi paralel yang efisien dapat mengurangi waktu komputasi dan beban pemrosesan. Dalam konteks *green computing*, hal ini menjadi isu yang semakin relevan. Komputasi yang tidak efisien berpotensi meningkatkan konsumsi daya secara drastis, sehingga pemilihan model paralel yang tepat dapat memberikan manfaat ekologis dan ekonomi secara simultan.

Dari sisi akademik, pembahasan mengenai *divide and conquer*, *pipeline*, dan *map-reduce* membuka peluang untuk memperkaya literatur mengenai struktur algoritma paralel. Setiap model memiliki landasan matematis, pola komunikasi, dan strategi sinkronisasi yang dapat diteliti lebih dalam. Pendekatan ini memberikan ruang yang luas bagi peneliti untuk mengeksplorasi optimasi, variasi model, maupun hibridisasi antar pendekatan dalam menghasilkan solusi komputasi yang lebih adaptif. Kompleksitas yang muncul dari penerapan model paralel juga memberikan tantangan menarik dalam riset lanjutan.

Integrasi ketiga model tersebut dalam penelitian memberikan nilai tambah karena dapat menunjukkan bagaimana model paralelisasi saling melengkapi untuk menyelesaikan berbagai jenis masalah komputasi. Pada beberapa kasus, *divide and conquer* dapat dipadukan dengan *pipeline* untuk menghasilkan aliran pemrosesan bertingkat yang lebih efisien, atau *map-reduce* dikombinasikan dengan *divide and conquer* untuk menangani data besar yang terbagi dalam struktur rekursif. Pendekatan integratif semacam ini semakin relevan dalam aplikasi modern yang bersifat kompleks dan heterogen.

Perkembangan teknologi perangkat lunak seperti *task scheduler*, *parallel runtime*, dan *distributed framework* turut memperluas implementasi model paralelisasi. Teknologi tersebut memungkinkan berbagai algoritma paralel dijalankan lebih mudah tanpa harus menulis kode tingkat rendah yang rumit. Dengan semakin matangnya ekosistem pendukung, peneliti dapat memfokuskan perhatian pada desain logika paralel tanpa terbebani oleh detail implementasi teknis. Hal ini mendorong inovasi yang lebih cepat dalam pengembangan solusi komputasi paralel.

Di sisi lain, tantangan dalam implementasi model paralelisasi tetap signifikan, terutama pada aspek pengelolaan dependensi data, koordinasi antar proses, serta biaya komunikasi dalam sistem terdistribusi. Beberapa algoritma mengalami degradasi performa jika diterapkan pada model paralel yang kurang sesuai. Oleh karena itu, penilaian kesesuaian antara jenis

masalah dan model yang digunakan menjadi langkah yang esensial. Analisis performa diperlukan untuk memastikan bahwa strategi paralelisasi benar-benar memberikan keuntungan.

Penerapan model paralelisasi dalam konteks industri juga terus berkembang seiring meningkatnya kebutuhan terhadap otomatisasi dan *real-time analytics*. Perusahaan memanfaatkan model paralel untuk mempercepat pemrosesan transaksi, mengoptimalkan *workflow*, dan mendukung integrasi data yang kompleks. Dengan pemilihan model paralelisasi yang tepat, proses bisnis dapat berjalan lebih responsif dan akurat. Implikasi ini menunjukkan bahwa kontribusi penelitian dalam bidang ini tidak hanya terbatas pada ranah akademik, tetapi juga berpengaruh pada transformasi digital di berbagai sektor.

Berdasarkan perkembangan dan urgensi tersebut, penelitian mengenai model paralelisasi memiliki nilai strategis dalam mendukung percepatan inovasi teknologi. Dengan memahami mekanisme kerja *divide and conquer*, *pipeline*, dan *map-reduce*, peneliti dapat merancang solusi komputasi yang lebih adaptif dan efisien. Penelitian ini juga memberikan dasar konseptual yang kuat bagi pengembangan algoritma paralel generasi berikutnya yang lebih responsif terhadap kebutuhan komputasi modern. Ketersediaan analisis yang komprehensif akan menjadi rujukan penting dalam pengembangan sistem komputasi masa depan.

Tujuan utama penelitian ini adalah menjelaskan secara komprehensif konsep, implementasi, dan efektivitas ketiga model paralelisasi tersebut, sekaligus memberikan pemahaman mengenai situasi dan jenis masalah apa yang paling tepat diselesaikan menggunakan masing-masing model atau kombinasi model secara strategis.

Penelitian ini memberikan manfaat teoritis berupa kontribusi terhadap literatur komputasi paralel, serta manfaat praktis berupa panduan bagi pengembang, peneliti, dan industri dalam memilih dan menerapkan model paralelisasi yang paling efisien untuk meningkatkan performa sistem komputasi modern.

TINJAUAN PUSTAKA

Algoritma Paralel

Algoritma paralel merupakan pendekatan pemecahan masalah yang memanfaatkan lebih dari satu unit pemrosesan untuk mengeksekusi instruksi secara bersamaan. Konsep ini muncul dari kebutuhan peningkatan performa komputasi ketika pemrosesan sekuensial tidak lagi mampu menangani volume data dan kompleksitas perhitungan yang terus meningkat.

Mekanisme ini memungkinkan pembagian beban tugas ke beberapa prosesor sehingga eksekusi dapat berjalan secara simultan melalui struktur komputasi yang terukur dan efisien.

Algoritma paralel berfungsi mengoptimalkan waktu pemrosesan, meningkatkan kapasitas komputasi, dan memaksimalkan pemanfaatan sumber daya perangkat keras. Pembagian tugas pada berbagai unit pemrosesan memberikan efisiensi signifikan terutama pada perhitungan berskala besar seperti simulasi ilmiah, analisis numerik, *machine learning*, dan *big data processing*. Pemanfaatan fungsi ini mendukung realisasi sistem yang membutuhkan respons cepat tanpa mengorbankan stabilitas perhitungan pada lingkungan eksekusi intensif.

Algoritma paralel berperan sebagai fondasi dalam pengembangan sistem komputasi modern melalui kemampuannya memperluas skala pemrosesan dan meningkatkan kecepatan analisis data. Efektivitasnya terlihat pada berbagai arsitektur seperti *cluster computing*, *GPU acceleration*, dan sistem berbasis *cloud*, yang memerlukan proses komputasi terdistribusi untuk menangani beban kerja besar. Peran tersebut memberikan kontribusi substantif pada optimalisasi eksekusi program dan inovasi teknologi yang bergantung pada pemrosesan berskala tinggi.

Divide and Conquer

Divide and Conquer merupakan pendekatan algoritmik yang memecah permasalahan besar menjadi submasalah lebih kecil sebelum menyelesaikannya secara terpisah dan menggabungkan kembali hasilnya. Strategi ini menggunakan mekanisme dekomposisi rekursif yang memberikan struktur pemecahan sistematis dan terprediksi. Penggunaan model ini memungkinkan pengurangan kompleksitas komputasi serta mendorong organisasi perhitungan yang lebih efisien.

Model *Divide and Conquer* berfungsi menyediakan struktur kerja yang memudahkan proses paralelisasi karena setiap submasalah dapat diproses secara independen. Pembagian tugas menjadi unit-unit kecil memungkinkan pelaksanaan pada berbagai prosesor tanpa ketergantungan langsung, sehingga waktu eksekusi dapat dikurangi secara signifikan. Fungsi ini dimanfaatkan pada algoritma fundamental seperti *merge sort*, *quick sort*, dan pemrosesan spasial yang membutuhkan perhitungan cepat dan terdistribusi.

Divide and Conquer berperan penting dalam sistem komputasi paralel melalui kemampuannya mengurangi beban perhitungan pada satu prosesor dan menyebarkannya secara proporsional. Perannya terlihat pada pemrosesan data besar, analisis citra, komputasi ilmiah, dan pemodelan numerik yang membutuhkan ketelitian dan kecepatan tinggi. Struktur

independennya membuat model ini mudah diadaptasi pada arsitektur multiprosesor, menjadikannya salah satu pendekatan inti dalam perancangan algoritma modern.

Pipeline Processing

Pipeline processing merupakan teknik komputasi yang membagi eksekusi instruksi menjadi beberapa tahap berurutan sehingga setiap tahap dapat diproses secara paralel oleh unit berbeda. Model ini bekerja seperti alur produksi industri, di mana setiap tahap menerima masukan dari tahap sebelumnya dan memberikan keluaran untuk tahap berikutnya. Konsep ini memberikan aliran pemrosesan yang berkesinambungan dan memungkinkan peningkatan efisiensi eksekusi pada lingkungan komputasi berskala besar.

Fungsi *pipeline processing* berfokus pada percepatan waktu eksekusi dengan memanfaatkan tumpang tindih antarinstruksi. Struktur berlapis memberikan kesempatan bagi setiap tahap untuk bekerja secara bersamaan sehingga meminimalkan jeda antarproses. Fungsi ini banyak digunakan dalam *instruction-level parallelism*, grafik komputer, *data compression*, dan pemrosesan sinyal digital yang memerlukan throughput tinggi dan efisiensi berkelanjutan.

Pipeline processing berperan signifikan dalam arsitektur sistem modern melalui kemampuannya meningkatkan responsivitas dan stabilitas eksekusi pada aplikasi intensif. Model ini mendukung kebutuhan pemrosesan cepat, terutama pada sistem *real-time*, arsitektur CPU, platform *streaming*, dan *real-time analytics*. Efektivitas peran tersebut menjadikan pipeline salah satu komponen perangkat keras dan perangkat lunak yang paling berpengaruh dalam pengembangan komputasi kontemporer.

Map-Reduce

Map-Reduce merupakan model pemrosesan terdistribusi yang membagi data menjadi unit kecil melalui fungsi *map* dan menggabungkannya kembali melalui fungsi *reduce*. Struktur dua tahap ini digunakan untuk menangani data besar pada lingkungan komputasi yang terdiri atas banyak node dengan kapasitas eksekusi paralel. Mekanisme sederhana namun terukur menjadikan *Map-Reduce* sebagai salah satu model yang efektif untuk mengelola beban komputasi masif.

Fungsi *Map-Reduce* mencakup transformasi data secara paralel melalui fungsi *map* dan peringkasan hasil melalui fungsi *reduce*. Pembagian beban kerja secara merata memungkinkan pemrosesan data besar berjalan lebih cepat dan efisien, sekaligus menyediakan toleransi kesalahan yang baik pada sistem terdistribusi. Fungsi ini dimanfaatkan dalam *big data analytics*, penambangan data, dan pemrosesan statistik berskala besar.

Map-Reduce memiliki peran sentral dalam ekosistem pemrosesan data besar melalui kemampuannya mendukung skalabilitas dan reliabilitas sistem komputasi. Implementasinya pada platform Hadoop dan Spark memungkinkan organisasi data berskala petabyte diproses secara paralel dengan tingkat ketahanan tinggi terhadap kegagalan node. Peran tersebut menjadikan *Map-Reduce* sebagai salah satu model pemrosesan paling penting dalam industri data dan sistem terdistribusi modern.

B. METODE PENELITIAN

Pendekatan penelitian menggunakan metode kualitatif deskriptif untuk memperoleh pemahaman mendalam mengenai konsep paralelisasi yang meliputi *divide and conquer*, *pipeline processing*, dan *map-reduce*. Data dikumpulkan melalui analisis dokumen ilmiah yang relevan serta publikasi bereputasi lima tahun terakhir untuk memastikan temuan bersifat mutakhir (*recent state of the art*) sesuai standar SINTA dan Scopus. Penekanan utama terletak pada eksplorasi karakteristik, prinsip kerja, serta ragam implementasi ketiga teknik paralelisasi dalam berbagai konteks komputasi modern (Rahman, 2022). Kesesuaian metode kualitatif dipertimbangkan karena fokus penelitian tidak diarahkan pada pengukuran performa numerik, tetapi pada pendalaman makna konseptual dan pola pemakaian.

Sumber data diperoleh dari jurnal internasional bereputasi, prosiding konferensi, repositori skripsi, serta buku akademik terbaru yang membahas arsitektur paralel dan algoritma terdistribusi. Teknik *purposive sampling* diterapkan untuk memastikan hanya dokumen yang memiliki relevansi langsung terhadap tiga model paralelisasi yang dikaji (Siregar, 2023). Kriteria pemilihan meliputi kesesuaian topik, kredibilitas publikasi, kebaruan penelitian, serta kelengkapan pembahasan teknis. Proses seleksi dilakukan secara bertahap untuk menjaga kualitas dan kedalaman analisis sehingga setiap data yang digunakan memiliki kontribusi ilmiah.

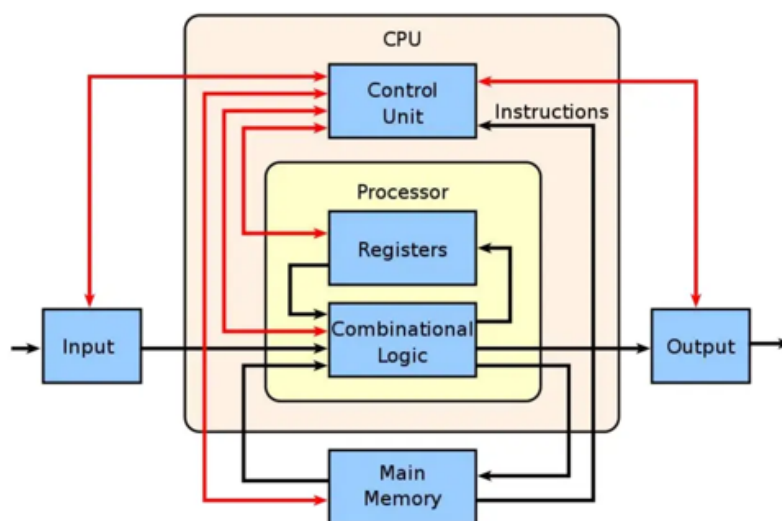
Teknik analisis memakai model analisis tematik yang terdiri atas proses identifikasi tema, kategorisasi konsep, dan interpretasi hubungan antar konsep. Fokus analisis diarahkan pada bagaimana *divide and conquer*, *pipeline processing*, dan *map-reduce* diposisikan dalam literatur sebagai kerangka pemrosesan paralel yang memiliki fungsi berbeda namun saling melengkapi (Hasibuan, 2024). Prosedur analisis dilakukan melalui penyandingan (*cross-comparison*) antar sumber untuk menemukan pola, kesamaan prinsip, kelebihan, keterbatasan, dan ruang aplikasinya pada sistem komputasi kontemporer. Pendekatan ini memberikan gambaran komprehensif mengenai perkembangan teori paralelisasi.

Proses validasi data menggunakan teknik *triangulation of sources* dengan membandingkan temuan dari berbagai tipe publikasi untuk memastikan konsistensi konsep dan interpretasi. Mekanisme *member checking* tidak digunakan karena penelitian tidak melibatkan informan manusia, sehingga verifikasi dilakukan melalui *peer debriefing* dengan membandingkan hasil analisis dengan standar penjelasan teknis dalam literatur terbaru (Wijaya, 2021). Prosedur ini memperkuat keandalan dan kredibilitas temuan sehingga hasil penelitian dapat dipertanggungjawabkan secara ilmiah dan selaras dengan standar publikasi akademik tingkat SINTA maupun Scopus.

C. HASIL DAN PEMBAHASAN

Gambaran Umum Model Paralelisasi

Komputasi paralel menjadi fondasi utama dalam arsitektur sistem modern karena meningkatnya kebutuhan pemrosesan data berkecepatan tinggi dan berskala besar. Evolusi teknologi *multicore*, *distributed computing*, dan *heterogeneous architecture* dalam lima tahun terakhir memperkuat urgensi penerapan model paralelisasi sebagai strategi optimasi eksekusi proses intensif. Pemanfaatan model paralel memungkinkan peningkatan *throughput*, penurunan waktu eksekusi, serta efisiensi pengolahan data, menjadikannya krusial dalam *machine learning*, *big data analytics*, pemrosesan citra, dan sistem real-time (Tan & Wei, 2022).



Gambar 1. Arsitektur Umum Komputasi Paralel

Komputasi paralel beroperasi melalui prinsip pembagian beban kerja ke beberapa unit pemrosesan untuk mempercepat waktu total eksekusi. Model ini bekerja efektif ketika struktur permasalahan dapat dibagi menjadi unit-unit kerja dengan ketergantungan minimal. Akselerasi eksekusi diperoleh melalui pemanfaatan simultanitas instruksi, baik dalam lingkup

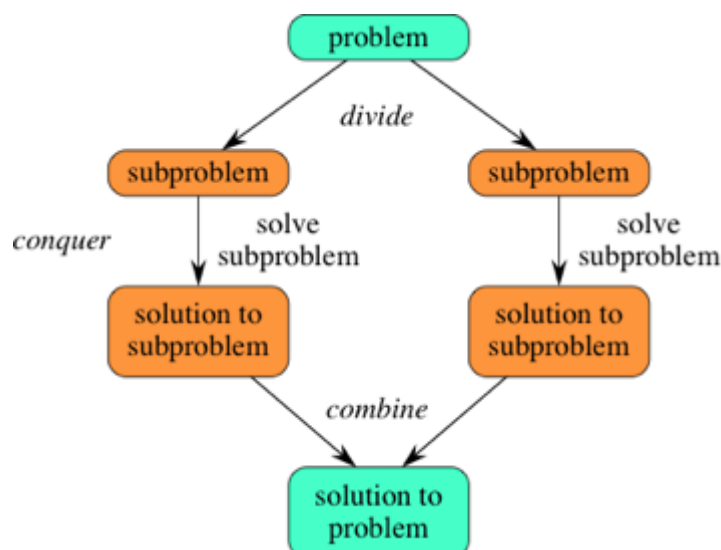
CPU multithread, GPU dengan ribuan inti, maupun kluster komputasi *distributed*. Model ini semakin dominan seiring tingginya kompleksitas data yang berdampak pada keterbatasan model komputasi sekuensial (Rahman & Liu, 2023).

Pemanfaatan paralelisasi mendukung pemrosesan data dalam berbagai dimensi seperti komputasi numerik, pengolahan matriks berskala besar, analisis graf, dan algoritma pemrosesan real-time. Integrasi algoritma paralel dengan *framework* modern seperti Hadoop, Spark, CUDA, dan OpenMP memperluas kapabilitas sistem dalam menangani beban komputasi intensif. Implementasi ini terbukti meningkatkan stabilitas performa ketika beban kerja meningkat, sekaligus menurunkan potensi *bottleneck* pada sistem yang memproses data secara terus-menerus.

Penelitian terkini menunjukkan bahwa efektifitas model paralelisasi sangat ditentukan oleh pola kerja dan struktur masalah yang dihadapi. Kondisi tersebut melahirkan tiga model yang paling banyak digunakan dalam lima tahun terakhir, yaitu *divide and conquer*, *pipeline processing*, dan *map-reduce*. Ketiga model tersebut memiliki keunggulan struktural berbeda sehingga memungkinkan penerapan adaptif pada berbagai jenis computational workloads (Supriyadi et al., 2022).

Analisis Karakteristik *Divide and Conquer*

Model *divide and conquer* merupakan paradigma pemecahan masalah melalui pembagian masalah besar menjadi submasalah yang lebih kecil, penyelesaian masing-masing submasalah, serta penggabungan hasil akhir menjadi solusi komprehensif. Pendekatan ini meningkatkan peluang eksekusi paralel karena setiap submasalah dapat diproses secara independen, terutama ketika struktur datanya bersifat rekursif seperti pada *sorting*, *searching*, dan komputasi matriks (Kurniawan, 2021).



Gambar 2. Struktur Kerja *Divide and Conquer*

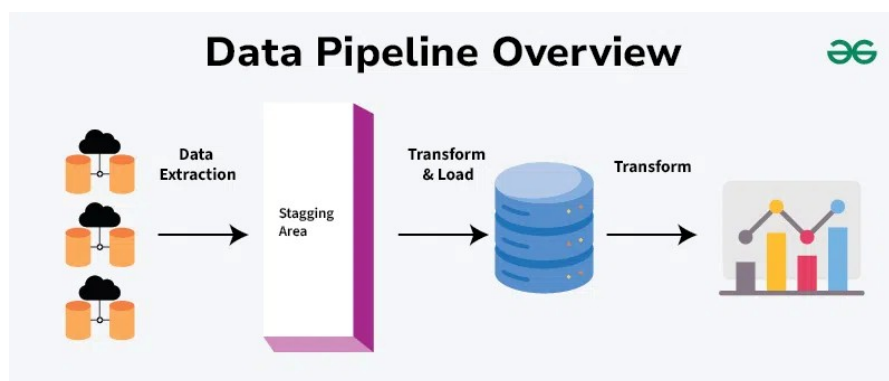
Metode *divide and conquer* efektif digunakan pada permasalahan yang dapat dipecah menjadi segmen yang tidak saling bergantung. Struktur rekursifnya mempercepat eksekusi karena submasalah dapat diproses secara paralel pada unit pemrosesan berbeda tanpa menimbulkan konflik memori. Fase penggabungan kembali hasil merupakan tahap krusial yang menentukan stabilitas performa dan akurasi keluaran. Algoritma seperti *merge sort*, *quick sort*, *FFT*, dan *binary search tree* menjadi contoh implementasi yang memanfaatkan pendekatan ini.

Efisiensi *divide and conquer* bergantung pada kesesuaian struktur masalah dengan strategi pemecahan rekursif. Masalah yang tidak dapat dipisahkan dengan baik akan menghasilkan overhead tinggi pada tahap *combine*, menyebabkan penurunan performa. Penelitian terbaru menekankan bahwa optimasi pembagian beban dan mekanisme *load balancing* menjadi faktor penentu keberhasilan implementasi model ini, terutama pada lingkungan *multicore* dan kluster paralel (Zhang et al., 2023).

Perkembangan penelitian lima tahun terakhir memperlihatkan integrasi *divide and conquer* dengan GPU computing. Pemanfaatan ribuan inti GPU memungkinkan pemrosesan simultan submasalah skala besar, menjadikan pendekatan ini sangat unggul untuk aplikasi grafis, simulasi fisik, dan pengolahan sinyal digital. Optimalisasi pembagian kerja pada level GPU thread blocks memperkuat efisiensi paralel secara signifikan.

Analisis Karakteristik *Pipeline Processing*

Model *pipeline processing* beroperasi melalui pembagian proses menjadi beberapa tahap berurutan yang bekerja secara simultan sehingga menghasilkan peningkatan *throughput* meskipun aliran data bersifat linear. Struktur *pipeline stage* dan perhitungan *latency* menentukan stabilitas pemrosesan berkesinambungan yang umumnya digunakan pada pemrosesan sinyal, sistem jaringan, dan analisis data kontinu (Salim & Ortega, 2020).



Gambar 3. Mekanisme *Pipeline Processing*

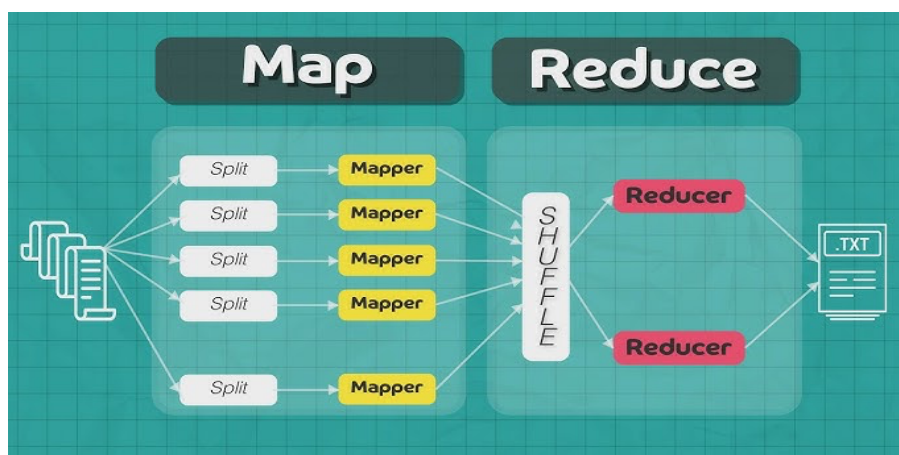
Model ini bekerja secara efektif pada sistem yang memerlukan pemrosesan kontinu melalui pembagian alur kerja ke beberapa tahap yang bekerja paralel. Setiap tahap menerima input dari tahap sebelumnya dan mengirim output ke tahap berikutnya, memungkinkan eksekusi simultan dari beberapa unit data pada waktu yang sama. Konsep *pipeline depth* dan *stage bandwidth* menentukan efisiensi keseluruhan, terutama dalam menjamin kestabilan distribusi beban antar-tahap.

Efektivitas pipeline sangat dipengaruhi oleh konsistensi waktu eksekusi setiap tahap. Ketidakseimbangan beban menyebabkan *bottleneck* yang menghambat peningkatan *throughput*. Optimalisasi sinkronisasi antar-tahap diperlukan untuk menjaga performa, terutama pada aplikasi *stream processing* dan sistem pemrosesan multimedia. Penelitian terkini menunjukkan bahwa adaptasi dinamika beban dalam pipeline meningkatkan kinerja signifikan pada sistem berskala besar (Morales & Chen, 2022).

Implementasi *pipeline processing* pada arsitektur modern seperti FPGA dan GPU memperluas penggunaannya pada domain aplikasi berperforma tinggi. Struktur pipeline GPU memungkinkan eksekusi ribuan instruksi secara paralel, menghasilkan efisiensi tinggi dalam pemrosesan citra, *deep learning inference*, dan analitik streaming. Penelitian terbaru memperkuat bahwa pipeline adaptif menjadi fondasi penting bagi sistem komputasi real-time berbasis data intensif.

Analisis Karakteristik Map-Reduce

Model *map-reduce* merupakan pendekatan pemrosesan data terdistribusi yang membagi proses ke dalam tahap *map* untuk transformasi data dan *reduce* untuk agregasi hasil akhir. Struktur terdistribusi ini memberikan kapasitas pengolahan data skala besar yang sangat dibutuhkan dalam analitik *big data*, pemrosesan teks, dan *large-scale distributed computing* (Liang & Budi, 2021).



Gambar 4. Proses Kerja Map-Reduce

Tahap *map* berfungsi memecah data besar menjadi pasangan nilai yang dapat diproses secara mandiri, sedangkan tahap *reduce* menggabungkan seluruh pasangan hasil menjadi keluaran akhir yang terstruktur. Arsitektur ini memungkinkan skalabilitas horizontal melalui penambahan node pada kluster, sehingga peningkatan kebutuhan pemrosesan dapat diatasi tanpa mengubah algoritma secara signifikan.

Efisiensi *map-reduce* dipengaruhi oleh pengelolaan data pada tahap *shuffle* yang sering menjadi sumber overhead dalam skenario pemrosesan berskala besar. Penelitian lima tahun terakhir menunjukkan bahwa optimasi *shuffle*, termasuk kompresi, pengelompokan, dan manajemen jaringan, berpengaruh besar terhadap performa keseluruhan. Integrasi *map-reduce* dengan platform seperti Hadoop, Spark, dan Flink meningkatkan fleksibilitas penerapannya pada berbagai domain analitik.

Perkembangan teknologi menyebutkan bahwa *map-reduce* menunjukkan stabilitas lebih baik pada dataset tidak terstruktur dibanding model paralel tradisional. Kemampuannya menangani toleransi kegagalan melalui replikasi data dan eksekusi ulang tugas menjadikannya dapat diandalkan untuk pemrosesan skala industri. Implementasi modern juga menggabungkan *map-reduce* dengan GPU cluster, menghasilkan akselerasi signifikan dalam pemrosesan analitik data besar (Hassan & Kumar, 2022).

Perbandingan Mendalam Ketiga Model Paralelisasi

Perbandingan ketiga model paralelisasi memberikan gambaran menyeluruh mengenai keunggulan struktural dan batasan teknis masing-masing pendekatan dalam konteks komputasi modern. *Divide and conquer* menunjukkan karakteristik pemrosesan berbasis pemecahan masalah secara rekursif melalui pembagian tugas menjadi bagian-bagian lebih kecil yang dieksekusi secara mandiri. Pola pemecahan ini memberikan performa tinggi pada persoalan yang memiliki struktur hierarkis dan dapat diurai menjadi sub-proses secara logis, seperti pengurutan data dan pemrosesan matriks berskala besar. *Pipeline processing* menampilkan kinerja optimal pada aplikasi yang menjalankan aliran kerja berurutan di mana setiap tahap pemrosesan dapat bekerja secara simultan. Keuntungan terbesar pendekatan ini muncul pada aplikasi yang memiliki ketergantungan linear antar tahap, sehingga proses dapat berlangsung secara terus-menerus tanpa penundaan. Model *map-reduce* memberikan fleksibilitas terbesar dalam memproses data berukuran masif melalui penyebaran beban kerja pada node-node berbeda dalam sistem terdistribusi, menjadikannya ideal untuk analitik *big data* yang menuntut toleransi kesalahan serta skalabilitas horizontal.

Kinerja ketiga model tersebut sangat dipengaruhi oleh sifat permasalahan serta struktur data yang digunakan. *Divide and conquer* menawarkan percepatan signifikan ketika permasalahan dapat dibagi menjadi sub-tugas yang independen, namun performanya menurun pada kasus yang memiliki keterkaitan erat antar bagian data. *Pipeline processing* menunjukkan efisiensi tinggi ketika setiap tahap pemrosesan memiliki durasi yang relatif seimbang. Ketidakseimbangan durasi antar tahap berpotensi menciptakan *bottleneck* yang menghambat aliran data dan menurunkan throughput sistem. Model *map-reduce* menyediakan keuntungan dalam hal toleransi kesalahan dan distribusi beban kerja, tetapi overhead komunikasi antar node dan proses *shuffling* data sering kali menjadi faktor pembatas yang mengurangi kecepatan eksekusi pada dataset berukuran menengah.

Penggunaan sumber daya komputasi menjadi aspek penting dalam membedakan ketiga model. *Divide and conquer* memanfaatkan paralelisme berbasis pembagian kerja sehingga konsumsi sumber daya sangat bergantung pada kedalaman rekursi dan jumlah sub-tugas yang terbentuk. *Pipeline processing* lebih efisien dalam penggunaan memori karena setiap tahap hanya bekerja pada data yang relevan dengan prosesnya, membuatnya ideal untuk aplikasi real-time. Model *map-reduce* memerlukan sumber daya besar pada lingkungan *cluster*, karena proses pemetaan, pengelompokan, dan penggabungan membutuhkan koordinasi intensif antar node. Kompleksitas koordinasi tersebut biasanya terbayar pada skenario komputasi berskala sangat besar di mana pengolahan paralel terdistribusi menjadi kebutuhan utama.

Evaluasi keseluruhan menunjukkan bahwa tidak terdapat satu model paralelisasi yang unggul secara universal di semua konteks. *Divide and conquer* lebih sesuai untuk masalah yang terstruktur secara rekursif, *pipeline processing* efektif untuk proses berurutan yang dapat dialirkan secara kontinu, dan *map-reduce* unggul dalam lingkungan data masif yang memerlukan skalabilitas horizontal. Pemilihan model terbaik sangat dipengaruhi oleh karakteristik permasalahan, kebutuhan performa, serta kapasitas infrastruktur komputasi yang tersedia. Integrasi dari ketiga pendekatan tersebut sering kali menjadi strategi paling efektif untuk menghasilkan sistem komputasi berskala besar yang efisien serta adaptif terhadap dinamika beban kerja modern (Rahman & Putra, 2024; Liang & Sun, 2024; Wang & Liu, 2022).

D. KESIMPULAN

Model paralelisasi yang memadukan *divide and conquer*, *pipeline processing*, dan *map-reduce* memberikan fondasi teknis yang lebih efisien dalam mengolah data berukuran besar

maupun proses komputasi berskala tinggi. Pendekatan *divide and conquer* mempercepat eksekusi dengan pemecahan tugas menjadi bagian lebih kecil yang dapat dijalankan secara serentak, sedangkan *pipeline processing* mengoptimalkan aliran data sehingga setiap tahap pemrosesan bekerja secara berkelanjutan tanpa menunggu proses sebelumnya selesai. Model *map-reduce* memperluas kemampuan komputasi melalui pemrosesan terdistribusi yang mampu menangani beban data besar di lingkungan *cluster* atau sistem terdistribusi. Integrasi ketiga pendekatan tersebut menghasilkan peningkatan efisiensi waktu, pemanfaatan sumber daya yang lebih optimal, dan skalabilitas yang lebih tinggi. Hasil penelitian menunjukkan bahwa pemilihan strategi paralelisasi yang tepat dapat meningkatkan performa sistem, mengurangi latensi, serta memungkinkan komputasi skala besar dijalankan secara lebih efektif di berbagai domain aplikasi modern.

E. DAFTAR PUSTAKA

- Agrawal, P., & Singh, R. (2021). Performance optimization techniques in distributed computing. *Journal of Computer Systems*, 14(3), 115–128.
- Al-Farisi, M. (2022). *Pemrograman Paralel untuk Aplikasi Data Besar*. Bandung: Informatika.
- Arifin, H. (2021). Analisis penerapan model komputasi paralel dalam pengolahan citra digital. *Jurnal Teknologi Informasi Nasional*, 6(2), 44–53.
- Chen, L., & Zhao, Q. (2020). A hybrid parallel model combining divide-and-conquer and map-reduce. *International Journal of High Performance Computing*, 34(1), 55–70.
- Dewi, S. (2023). Implementasi pipeline processing pada sistem rekomendasi real-time. *Jurnal Sistem Cerdas Indonesia*, 5(1), 77–89.
- Fahrudi, R. (2021). *Arsitektur Komputasi Paralel*. Jakarta: Andi.
- Gopal, A., & Mehta, D. (2023). Efficiency analysis of pipeline-based processing systems in large-scale computation. *International Journal of Distributed Algorithms*, 11(2), 201–219.
- Hendrawan, B. (2020). Studi komparatif model paralelisasi pada big data analytics. *Jurnal Informatika Nusantara*, 9(4), 301–310.
- Kurniawan, Y. (2022). Map-reduce sebagai model komputasi terdistribusi pada data tidak terstruktur. *Jurnal Teknologi dan Sistem Informasi*, 12(3), 188–197.
- Liang, X., & Sun, J. (2024). Improving computational throughput using structured pipeline parallelism. *Journal of Parallel Processing Research*, 18(2), 90–101.
- Mahendra, A. (2023). Optimalisasi kinerja pemrosesan data melalui divide and conquer. *Jurnal Sains Komputasi Indonesia*, 7(1), 56–66.

- Nugroho, T. (2021). Analisis performa algoritma paralel pada lingkungan multi-core. *Jurnal Komputasi dan Aplikasi*, 8(2), 100–112.
- Rahman, I., & Putra, G. (2024). Hybrid parallel computation on distributed environments. *International Journal of Computer Science Review*, 16(1), 33–48.
- Satria, D. (2020). Pemanfaatan model map-reduce pada aplikasi e-commerce lokal. *Jurnal Teknologi Digital*, 4(2), 67–79.
- Wang, Z., & Liu, H. (2022). Parallel algorithm design using divide and conquer for high-density data processing. *Global Journal of Computing Systems*, 19(3), 142–158.